Module 2 — HTML5

> **Objectif** : structurer le contenu d'une page web avec les balises sémantiques HTML5, créer des liens, images, listes, tableaux, formulaires et respecter les bonnes pratiques d'accessibilité et de SEO.

1. Qu'est-ce que HTML ?

HTML = *HyperText Markup Language*. C'est un **langage de balisage**, **pas** un langage de programmation : il ne contient ni variables ni boucles. Il sert à **structurer** le contenu d'une page : "ceci est un titre", "ceci est un paragraphe", "ceci est une image".

	HTML
Type	Langage de balisage
Créé en	1991 (Tim Berners-Lee)
Version actuelle	HTML5 (standard vivant, *living standard*)
Extension de fichier	`.html` (ou `.htm`)
Sensible à la casse ?	Non (mais on écrit tout en minuscules par convention)

2. Anatomie d'une balise

```
<p class="intro">Bonjour le monde</p>
└┘ └──┬──┘      └──────┬──────┘ └┘
 │     │               │        │
 │  Attribut         Contenu   Balise fermante
Balise ouvrante
```

- **Balise** : mot-clé entre `<` et `>` qui définit le type d'élément.
- **Attribut** : information complémentaire (`class`, `id`, `src`, `href`…). Toujours dans la balise ouvrante.
- **Contenu** : ce qui s'affiche entre les balises ouvrante et fermante.

Balises auto-fermantes (*void elements*)

Certaines balises n'ont pas de contenu et **ne se ferment pas** :

```
<img src="photo.jpg" alt="Une photo">
<br>
<hr>
<input type="text">
<meta charset="utf-8">
```

⚠️ En XHTML on écrivait `<br />` . En HTML5, c'est inutile (mais accepté).

3. Structure d'une page HTML5

Toute page HTML5 commence par ce squelette **incompressible** :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Titre de l'onglet</title>
  </head>
  <body>
    <!-- Contenu visible -->
  </body>
</html>
```

Élément	Rôle
`<!DOCTYPE html>`	Déclare que le document est du HTML5 (obligatoire en première ligne)
`<html lang="fr">`	Racine du document, langue principale
`<head>`	Métadonnées (invisibles à l'écran) : titre, encodage, liens CSS…
`<meta charset="utf-8">`	Encodage des caractères — indispensable pour les accents
`<meta name="viewport"…>`	Adaptation mobile (responsive)
`<title>`	Titre affiché dans l'onglet du navigateur et dans Google

Élément	Rôle
`<body>`	Contenu visible de la page

💡 **Raccourci VS Code** : tapez `!` puis Tab dans un fichier `.html` pour générer ce squelette automatiquement (Emmet).

4. Les balises de texte

Titres

Six niveaux, du plus important au moins important.

```
<h1>Titre principal de la page (un seul par page !)</h1>
<h2>Titre de section</h2>
<h3>Sous-section</h3>
<h4>…</h4>
<h5>…</h5>
<h6>Niveau le plus fin</h6>
```

⚠️ **Règle d'or** : un seul `<h1>` par page (le titre principal). N'utilisez **jamais** les titres pour leur taille — utilisez CSS pour ça.

Paragraphe

```
<p>Un paragraphe de texte. Tout retour à la ligne ou espace multiple est ignoré par HTML.</p>
```

Mise en forme inline

Balise	Sens **sémantique**	Apparence par défaut
`<strong>`	Importance forte	**Gras**
`<em>`	Mise en emphase	*Italique*
`<mark>`	Surligné	fond jaune
`<small>`	Texte secondaire (mentions légales…)	Plus petit
`<s>`	Texte invalide / barré	~~Barré~~

Balise	Sens sémantique	Apparence par défaut
`<sub>` / `<sup>`	Indice / exposant	H_2O / x^2
`<code>`	Code informatique inline	`monospace`
`<kbd>`	Touche du clavier	Ctrl+C
`<abbr title="…">`	Abréviation	Survol révèle la signification

⚠️ `<b>` et `<i>` existent encore mais sont **purement visuels** (sans sens). Préférez `<strong>` et `<em>`.

Sauts et séparations

```
<br>     <!-- Saut de ligne dans un paragraphe -->
<hr>     <!-- Ligne horizontale (changement de thème) -->
```

Citations

```
<blockquote cite="https://source.com">
  Une longue citation, en bloc.
</blockquote>

<p>Comme disait <q>Untel</q>, la citation courte est en ligne.</p>
```

5. Les listes

Liste non ordonnée (puces)

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Liste ordonnée (numérotée)

```
<ol>
  <li>Allumer l'ordinateur</li>
  <li>Ouvrir le navigateur</li>
  <li>Taper l'URL</li>
</ol>
```

Liste de définitions

```
<dl>
  <dt>HTML</dt>
  <dd>Langage de structure des pages web.</dd>
  <dt>CSS</dt>
  <dd>Langage de mise en forme.</dd>
</dl>
```

Les listes peuvent être **imbriquées** :

```
<ul>
  <li>Front-end
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
  <li>Back-end</li>
</ul>
```

6. Les liens

```
<!-- Lien externe -->
<a href="https://wikipedia.org">Wikipedia</a>

<!-- Lien interne (vers une autre page du même site) -->
<a href="contact.html">Contact</a>

<!-- Lien vers une ancre (section de la page) -->
<a href="#section-3">Aller à la section 3</a>

<!-- Lien e-mail -->
<a href="mailto:contact@exemple.com">Nous écrire</a>

<!-- Lien téléphone -->
<a href="tel:+22890000000">Nous appeler</a>

<!-- Ouvrir dans un nouvel onglet -->
<a href="https://exemple.com" target="_blank" rel="noopener noreferrer">Site externe</a>
```

Attribut	Rôle
`href`	Cible du lien (obligatoire)
`target="_blank"`	Ouvre dans un nouvel onglet
`rel="noopener noreferrer"`	Sécurité quand on utilise `target="_blank"`
`download`	Force le téléchargement au lieu de l'affichage
`title`	Info-bulle au survol

Chemins relatifs vs absolus

```
<a href="https://exemple.com/page.html">  <!-- Absolu -->
<a href="/cours/page.html">                <!-- Racine du site -->
<a href="page.html">                       <!-- Même dossier -->
<a href="../page.html">                    <!-- Dossier parent -->
<a href="./sous-dossier/page.html">        <!-- Sous-dossier -->
```

7. Les images

```
<img src="photo.jpg" alt="Description textuelle de l'image" width="800" height="600">
```

Attribut	Rôle
`src`	Chemin vers l'image (obligatoire)
`alt`	**Texte alternatif** (obligatoire pour l'accessibilité et le SEO)
`width` / `height`	Dimensions en pixels (évite le saut visuel au chargement)
`loading="lazy"`	Charge l'image seulement quand on s'en approche en scrollant

Formats d'images du Web

Format	Quand l'utiliser ?	Compression
JPEG (`.jpg`)	Photos, dégradés	Avec perte
PNG	Logos, captures, transparence	Sans perte
GIF	Animations simples	Sans perte (palette limitée)
SVG	Icônes, illustrations vectorielles	Vectoriel (zoomable à l'infini)
WebP	Remplaçant moderne de JPEG/PNG (plus léger)	Avec ou sans perte
AVIF	Format de pointe (encore plus léger)	Avec ou sans perte

Image responsive avec `<picture>`

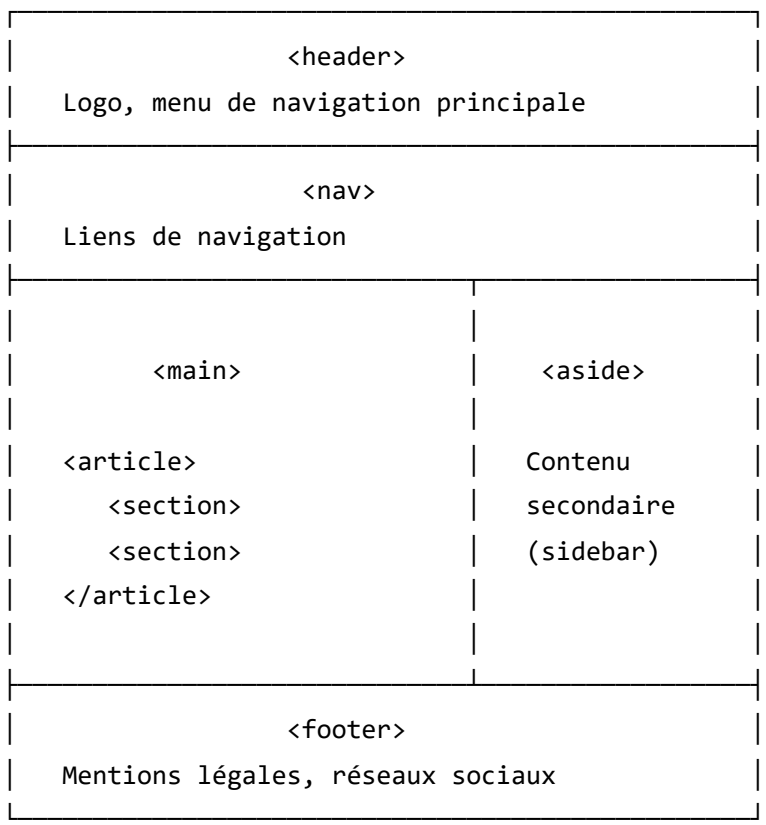
```
<picture>
  <source media="(max-width: 600px)" srcset="photo-mobile.jpg">
  <source media="(max-width: 1200px)" srcset="photo-tablette.jpg">
  <img src="photo-desktop.jpg" alt="Vue panoramique" loading="lazy">
</picture>
```

Figure et légende

```
<figure>
  <img src="schema.png" alt="Schéma du modèle client-serveur">
  <figcaption>Figure 1 — Échanges HTTP entre client et serveur.</figcaption>
</figure>
```

8. Les balises sémantiques HTML5

Avant HTML5, on utilisait `<div>` partout. HTML5 a introduit des balises qui **ont un sens** : meilleur pour l'accessibilité, le SEO et la maintenance.



Balise	Rôle
`<header>`	En-tête (de la page ou d'une section)
`<nav>`	Bloc de navigation
`<main>`	Contenu principal **unique** de la page
`<article>`	Contenu autonome, qui aurait du sens sorti du contexte (article de blog, fiche produit)

Balise	Rôle
`<section>`	Section thématique d'un document
`<aside>`	Contenu lié mais secondaire (encadré, pub)
`<footer>`	Pied (de page ou de section)
`<figure>` / `<figcaption>`	Image avec légende
`<time datetime="2026-05-18">`	Date/heure exploitable par machines
`<address>`	Coordonnées de contact

Exemple complet

```
<body>
  <header>
    <h1>Mon blog</h1>
    <nav>
      <ul>
        <li><a href="/">Accueil</a></li>
        <li><a href="/blog">Blog</a></li>
        <li><a href="/contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <header>
        <h2>Mon premier article</h2>
        <time datetime="2026-05-18">18 mai 2026</time>
      </header>
      <p>Contenu de l'article…</p>
      <footer>Par AGBAVON Kokou Bienvenu</footer>
    </article>
  </main>

  <aside>
    <h3>Articles populaires</h3>
    <ul>…</ul>
  </aside>

  <footer>
    <p>© 2026 — Tous droits réservés.</p>
  </footer>
</body>
```

9. Les tableaux

⚠️ Les tableaux servent à **présenter des données tabulaires**, **pas à mettre en page** (utilisez CSS Grid/Flexbox pour ça).

```html
<table>
  <caption>Notes du semestre</caption>
  <thead>
    <tr>
      <th>Matière</th>
      <th>Note</th>
      <th>Coefficient</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>HTML/CSS</td>
      <td>16</td>
      <td>2</td>
    </tr>
    <tr>
      <td>JavaScript</td>
      <td>14</td>
      <td>3</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td>Moyenne</td>
      <td colspan="2">14.8</td>
    </tr>
  </tfoot>
</table>
```

Balise	Rôle
`<table>`	Conteneur du tableau
`<caption>`	Titre du tableau
`<thead>` / `<tbody>` / `<tfoot>`	En-tête / corps / pied
`<tr>`	*Table Row* — ligne
`<th>`	*Table Header* — cellule d'en-tête
`<td>`	*Table Data* — cellule de données
`colspan="2"`	Fusion sur 2 colonnes
`rowspan="2"`	Fusion sur 2 lignes

10. Les formulaires

Les formulaires permettent de **collecter** des données saisies par l'utilisateur.

Structure de base

```
<form action="/traitement.php" method="post">
  <label for="prenom">Prénom :</label>
  <input type="text" id="prenom" name="prenom" required>

  <button type="submit">Envoyer</button>
</form>
```

Attribut de `<form>`	Rôle
`action`	URL qui reçoit les données
`method`	`get` (paramètres dans l'URL) ou `post` (dans le corps de la requête)
`enctype="multipart/form-data"`	Obligatoire pour envoyer des fichiers

Les types de champ `<input>`

`type`	Usage
`text`	Texte court sur une ligne
`password`	Mot de passe (masqué)
`email`	Adresse e-mail (validation automatique)
`tel`	Numéro de téléphone
`url`	URL
`number`	Nombre (avec flèches)
`range`	Curseur
`date`	Sélecteur de date
`time`	Sélecteur d'heure
`color`	Sélecteur de couleur

type	Usage
`checkbox`	Case à cocher
`radio`	Bouton radio (choix unique dans un groupe)
`file`	Téléversement de fichier
`hidden`	Champ caché
`submit`	Bouton d'envoi
`reset`	Bouton de réinitialisation

Autres champs

```
<!-- Texte long -->
<textarea name="message" rows="5" cols="40"></textarea>

<!-- Liste déroulante -->
<select name="pays">
  <option value="">-- Choisir --</option>
  <option value="tg">Togo</option>
  <option value="ci">Côte d'Ivoire</option>
  <option value="sn">Sénégal</option>
</select>

<!-- Groupe de cases -->
<fieldset>
  <legend>Centres d'intérêt</legend>
  <label><input type="checkbox" name="ci" value="web"> Web</label>
  <label><input type="checkbox" name="ci" value="mobile"> Mobile</label>
</fieldset>
```

Attributs de validation

Attribut	Rôle
`required`	Champ obligatoire
`minlength` / `maxlength`	Longueur min/max d'un texte
`min` / `max`	Valeur min/max d'un nombre/date
`pattern="[A-Za-z]+"`	Expression régulière de validation
`placeholder`	Texte gris d'aide (disparaît à la saisie)

Attribut	Rôle
`autocomplete`	`on` / `off` / `email` / `current-password` …
`disabled`	Champ désactivé
`readonly`	Champ lecture seule

Exemple complet de formulaire de contact

```
<form action="/contact" method="post">
  <fieldset>
    <legend>Vos coordonnées</legend>

    <div>
      <label for="nom">Nom complet *</label>
      <input type="text" id="nom" name="nom" required minlength="2">
    </div>

    <div>
      <label for="email">E-mail *</label>
      <input type="email" id="email" name="email" required
             placeholder="vous@exemple.com">
    </div>

    <div>
      <label for="sujet">Sujet</label>
      <select id="sujet" name="sujet">
        <option>Question générale</option>
        <option>Demande de devis</option>
        <option>Support technique</option>
      </select>
    </div>

    <div>
      <label for="message">Message *</label>
      <textarea id="message" name="message" rows="6" required></textarea>
    </div>
  </fieldset>

  <button type="submit">Envoyer le message</button>
</form>
```

💡 **Toujours associer `<label>` et `<input>`** via `for` / `id` : meilleure accessibilité, cliquer sur le label sélectionne le champ.

11. Médias : audio et vidéo

```
<!-- Vidéo -->
<video controls width="640" poster="apercu.jpg">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Votre navigateur ne supporte pas la vidéo HTML5.
</video>

<!-- Audio -->
<audio controls>
  <source src="musique.mp3" type="audio/mpeg">
  <source src="musique.ogg" type="audio/ogg">
</audio>

<!-- Vidéo YouTube intégrée -->
<iframe width="560" height="315"
        src="https://www.youtube.com/embed/VIDEO_ID"
        title="Titre de la vidéo"
        frameborder="0"
        allowfullscreen></iframe>
```

Attribut `<video>` / `<audio>`	Rôle
`controls`	Afficher les commandes (play, pause, volume…)
`autoplay`	Démarrer automatiquement (souvent bloqué par les navigateurs)
`loop`	Lire en boucle
`muted`	Démarrer sans son
`poster`	Image affichée avant lecture

12. Métadonnées et SEO

Le `<head>` contient des informations cruciales pour Google et les réseaux sociaux.

```html
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <title>Portfolio — Kokou Bienvenu AGBAVON</title>
  <meta name="description" content="Portfolio d'un développeur web full-stack basé à Lomé.">
  <meta name="keywords" content="portfolio, développeur, web, Lomé, Togo">
  <meta name="author" content="Kokou Bienvenu AGBAVON">

  <!-- Favicon -->
  <link rel="icon" href="favicon.ico">

  <!-- Open Graph (partage Facebook, LinkedIn) -->
  <meta property="og:title" content="Mon portfolio">
  <meta property="og:description" content="Découvrez mes projets web.">
  <meta property="og:image" content="https://exemple.com/preview.jpg">
  <meta property="og:url" content="https://exemple.com">

  <!-- Twitter Card -->
  <meta name="twitter:card" content="summary_large_image">

  <!-- CSS -->
  <link rel="stylesheet" href="style.css">

  <!-- JS (mieux à la fin du body en pratique) -->
  <script src="script.js" defer></script>
</head>
```

13. Accessibilité (a11y)

Quelques règles à appliquer systématiquement.

Règle	Pourquoi ?
`lang="fr"` sur `<html>`	Les lecteurs d'écran utilisent la bonne prononciation
`alt` sur **toutes** les images	Décrit l'image pour les non-voyants
Hiérarchie correcte des titres `h1` → `h6`	Permet de naviguer rapidement
Contraste suffisant texte/fond	Lisibilité pour daltoniens et malvoyants
`<label>` associé à chaque `<input>`	Cliquer sur le texte sélectionne le champ

Règle	Pourquoi ?
Navigation au clavier (Tab) possible	Pour ceux qui n'utilisent pas la souris
Utilisation des balises sémantiques	`<button>` plutôt que `<div onclick>`
Attributs ARIA si nécessaire	`aria-label`, `aria-hidden`, `role` ...

Test rapide

- Naviguez sur votre page avec la touche **Tab** uniquement. Tous les éléments interactifs doivent être atteignables.
- Auditez avec **Lighthouse** (DevTools → onglet *Lighthouse* → cocher *Accessibility*).

14. Commentaires

```
<!-- Ceci est un commentaire HTML, invisible pour l'utilisateur -->
<!--
  Commentaire
  sur plusieurs
  lignes.
-->
```

⚠️ Les commentaires HTML sont **visibles** dans le code source (clic droit → Afficher la source). N'y mettez jamais d'informations sensibles.

15. Caractères spéciaux (entités HTML)

Certains caractères ont une signification en HTML et doivent être **échappés**.

Caractère	Entité	Usage
`<`	`<`	Afficher un signe inférieur littéral
`>`	`>`	Afficher un signe supérieur littéral
`&`	`&`	Afficher une esperluette
`"`	`"`	Guillemet droit

Caractère	Entité	Usage
'	`'`	Apostrophe
(espace insécable)	` `	Espace qui empêche un saut de ligne
©	`©`	Copyright
€	`€`	Euro

16. Validation du code

Le **W3C Validator** vérifie que votre HTML est conforme au standard.

🔗 https://validator.w3.org

Un code valide est plus stable, mieux référencé et plus accessible.

17. Bonnes pratiques récapitulatives

✅ À faire

- Toujours commencer par `<!DOCTYPE html>` et `lang="fr"` .
- Indenter le code (2 ou 4 espaces) et utiliser des minuscules.
- Utiliser les balises sémantiques (`<header>` , `<nav>` , `<main>` …).
- Mettre des `alt` significatifs sur les images.
- Un seul `<h1>` par page.
- Associer `<label>` aux `<input>` .
- Valider avec le W3C Validator.

❌ À éviter

- Mettre en page avec des tableaux.
- Utiliser `<b>` , `<i>` , `<u>` , `<font>` pour le style (utilisez CSS).
- Oublier `alt` ou mettre `alt=""` partout.
- Empiler les `<div>` quand une balise sémantique existe.
- Mettre du CSS inline (`style="…"`) partout.

18. Mini-projet d'application

Créer une page `cv.html` qui présente votre CV avec :

- Un `<header>` contenant nom, photo, accroche.
- Un `<nav>` avec ancres vers les sections.
- Des `<section>` : *À propos*, *Compétences*, *Expériences*, *Formation*, *Projets*, *Contact*.
- Une liste de compétences.
- Un tableau des expériences (poste, entreprise, période).
- Un formulaire de contact.
- Un `<footer>` avec les réseaux sociaux.

→ Page valide W3C, score Lighthouse > 90 en accessibilité.

Ressources

- 📚 MDN — Apprendre HTML
- 📚 HTML5 Doctor — décisions sémantiques
- 📚 W3C Validator
- 📚 Can I Use
- 📖 *HTML & CSS — Design and Build Websites*, Jon Duckett (2011) — référence visuelle

➡️ **Module suivant** : 03 — CSS3